

Crear LLM





Índice

Práctica	2
Paso 1: Activación del entorno y preparación del framework Axolotl	2
Paso 2: Creación del archivo de datos para entrenar el modelo	3
Paso 3: Archivo de configuración del entrenamiento	3
Paso 4: Descarga de llama.cpp y aviso sobre el sistema de compilación	4
Paso 5: Instalación de dependencias desde requirements.txt	5
Paso 6: Descarga del modelo desde Hugging Face con Git LFS	6
Paso 7: Descarga del modelo desde Hugging Face con Git LFS	7
Paso 8: Configuración del modelo en la interfaz de ejecución	7
Paso 9: Archivo final del modelo convertido a GGUF	9
Paso 10: Prueba del modelo funcionando en la interfaz de chat	9

Práctica

Paso 1: Activación del entorno y preparación del framework Axolotl

La captura muestra que el entorno virtual venv ya está activo. Esto se identifica por el prefijo (venv) al inicio de cada línea de la terminal. A partir de este punto, todo lo que se instale o ejecute quedará aislado dentro de este entorno, garantizando que el proyecto utilice versiones controladas de cada librería.

Con el entorno activo, se ejecuta la instalación del framework Axolotl directamente desde su repositorio oficial. Este procedimiento asegura que se está utilizando una versión concreta y estable del proyecto, algo importante cuando se trabaja con herramientas en desarrollo activo.

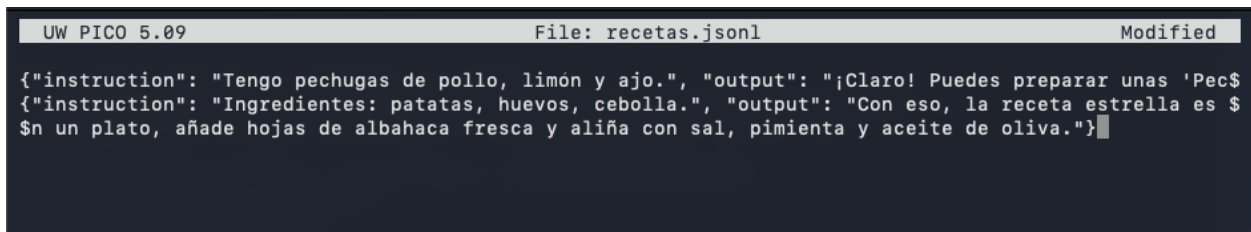
Cada una de estas librerías se instala automáticamente, dejando el entorno completamente equipado para los siguientes pasos del proyecto.

```
((venv) aulaateca26@Mac-mini-de-aulaateca26 mi-chef-bot % rm -rf venv
((venv) aulaateca26@Mac-mini-de-aulaateca26 mi-chef-bot % python3.11 -m venv venv
((venv) aulaateca26@Mac-mini-de-aulaateca26 mi-chef-bot % source venv/bin/activate
((venv) aulaateca26@Mac-mini-de-aulaateca26 mi-chef-bot % pip install "axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl"
Collecting axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl
  Cloning https://github.com/OpenAccess-AI-Collective/axolotl to /private/var/folders/x7/5vw0c3rx46j0bhmp88zdy3zm000gn/T/pip-install-0o_ika1v/axolotl_c6c119a320f94583bc3134b9ecc3fdd0
  Running command git clone --filter=blob:none --quiet https://github.com/OpenAccess-AI-Collective/axolotl /private/var/folders/x7/5vw0c3rx46j0bhmp88zdy3zm000gn/T/pip-install-0o_ika1v/axolotl_c6c119a320f94583bc3134b9ecc3fdd0
  Resolved https://github.com/OpenAccess-AI-Collective/axolotl to commit 236dad3bb7954e5dc8dcca5b4d067a7e6e39fb7e
    Installing build dependencies ... done
    Getting requirements to build wheel ... done
    Preparing metadata (pyproject.toml) ... done
Collecting packaging==26.0 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached packaging-26.0-py3-none-any.whl.metadata (3.3 kB)
Collecting huggingface_hub>=1.1.7 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached huggingface_hub-1.4.0-py3-none-any.whl.metadata (13 kB)
Collecting peft>=0.18.1 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached peft-0.18.1-py3-none-any.whl.metadata (14 kB)
Collecting tokenizers>=0.22.1 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached tokenizers-0.22.2-cp39-abi3-macosx_11_0_arm64.whl.metadata (7.3 kB)
Collecting transformers==5.0.0 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached transformers-5.0.0-py3-none-any.whl.metadata (37 kB)
Collecting accelerate==1.12.0 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached accelerate-1.12.0-py3-none-any.whl.metadata (19 kB)
Collecting datasets==4.5.0 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached datasets-4.5.0-py3-none-any.whl.metadata (19 kB)
Collecting trl==0.27.1 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached trl-0.27.1-py3-none-any.whl.metadata (11 kB)
Collecting hf_xet==1.2.0 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached hf_xet-1.2.0-cp37-abi3-macosx_11_0_arm64.whl.metadata (4.9 kB)
Collecting kernels==0.11.5 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached kernels-0.11.5-py3-none-any.whl.metadata (3.3 kB)
Collecting trackio>=0.13.0 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached trackio-0.15.0-py3-none-any.whl.metadata (9.9 kB)
Collecting typing_extensions>=4.15.0 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting optimum==1.16.2 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached optimum-1.16.2-py3-none-any.whl.metadata (17 kB)
Collecting hf_transfer (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Using cached hf_transfer-0.1.9-cp38-abi3-macosx_11_0_arm64.whl.metadata (1.7 kB)
Collecting sentencepiece (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
  Downloading sentencepiece-0.2.1-cp311-cp311-macosx_11_0_arm64.whl.metadata (10 kB)
Collecting gradio<7.0,>=6.2.0 (from axolotl @ git+https://github.com/OpenAccess-AI-Collective/axolotl)
```

Paso 2: Creación del archivo de datos para entrenar el modelo

En esta captura se muestra el archivo recetas.jsonl. Aquí se están definiendo los ejemplos que formarán parte del dataset con el que se entrenará el modelo. Cada línea del archivo contiene una instrucción y una respuesta en formato JSONL, un estilo muy utilizado para entrenar modelos de lenguaje porque permite almacenar ejemplos independientes de forma sencilla.

El contenido visible corresponde a ejemplos de consultas culinarias en español, donde el usuario proporciona ingredientes y el modelo debe generar una receta o sugerencia de cocina. Aunque las líneas aparecen cortadas por el tamaño de la ventana, se aprecia claramente que el archivo está siendo construido manualmente para enseñar al modelo a responder de manera útil dentro del dominio gastronómico.



```
UW PICO 5.09 File: recetas.jsonl Modified
{"instruction": "Tengo pechugas de pollo, limón y ajo.", "output": "¡Claro! Puedes preparar unas 'Pec$
{"instruction": "Ingredientes: patatas, huevos, cebolla.", "output": "Con eso, la receta estrella es $
$n un plato, añade hojas de albahaca fresca y aliña con sal, pimienta y aceite de oliva."}
```

Paso 3: Archivo de configuración del entrenamiento

Este archivo define todos los parámetros que utilizará Axolotl para entrenar el modelo de lenguaje. Aquí se especifica el modelo base que se va a ajustar, el tipo de tokenizador, el formato del dataset y las opciones de entrenamiento adaptadas a un equipo Mac de bajo consumo.

```
UW PICO 5.09 File: config_chef.yml Modified

base_model: microsoft/Phi-3-mini-4k-instruct
model_type: PhiForCausalLM
tokenizer_type: AutoTokenizer

# Configuración para bajo consumo (Mac)
load_in_4bit: false # Mac maneja mejor FP16 o BF16 que 4bit en entrenamiento local
load_in_8bit: false
bf16: false
fp16: false # El Mac mini usará float32 por defecto para mayor estabilidad
tf32: false
flash_attention: false
local_rank: 0
adapter: lora

datasets:
- path: recetas.jsonl
  type: alpaca

dataset_prepared_path: last_run_prepared
val_set_size: 0.05
sequence_len: 256 # Bajamos esto de 1024 a 256 para que tu RAM no sufra
sample_packing: true

lora_r: 8 # Reducimos la complejidad del adaptador
lora_alpha: 16
lora_dropout: 0.05
lora_target_modules:
- qkv_proj

num_epochs: 2 # Menos vueltas para terminar rápido
micro_batch_size: 1
gradient_accumulation_steps: 1
gradient_checkpointing: true
learning_rate: 0.0002
optimizer: adamw_torch
```

Paso 4: Descarga de llama.cpp y aviso sobre el sistema de compilación

En esta captura se muestra cómo, desde la terminal y dentro del entorno virtual, se clona el repositorio llama.cpp, una herramienta que permite ejecutar modelos de lenguaje de forma optimizada en CPU. Tras listar los archivos del proyecto, se descarga el repositorio completo desde GitHub y se accede a su carpeta.

Al intentar compilarlo con el comando make, aparece un mensaje importante: el proyecto ha dejado de utilizar Makefile como sistema de construcción y ahora requiere CMake. La terminal detiene el proceso y muestra un aviso indicando que las instrucciones actualizadas se encuentran en la documentación oficial del repositorio.

```

[(venv) aulaateca26@Mac-mini-de-aulaateca26 mi-chef-bot % ls
config_chef.yml recetas.jsonl venv
[(venv) aulaateca26@Mac-mini-de-aulaateca26 mi-chef-bot % git clone https://github.com/ggerganov/llama.
cpp
Clonando en 'llama.cpp'...
remote: Enumerating objects: 78196, done.
remote: Counting objects: 100% (89/89), done.
remote: Compressing objects: 100% (65/65), done.
remote: Total 78196 (delta 53), reused 24 (delta 24), pack-reused 78107 (from 3)
Recibiendo objetos: 100% (78196/78196), 285.39 MiB | 44.84 MiB/s, listo.
Resolviendo deltas: 100% (56583/56583), listo.
[(venv) aulaateca26@Mac-mini-de-aulaateca26 mi-chef-bot % cd llama.cpp
[(venv) aulaateca26@Mac-mini-de-aulaateca26 llama.cpp % make
Makefile:6: *** Build system changed:
The Makefile build has been replaced by CMake.

For build instructions see:
https://github.com/ggml-org/llama.cpp/blob/master/docs/build.md

. Stop.
[(venv) aulaateca26@Mac-mini-de-aulaateca26 llama.cpp % ]

```

Paso 5: Instalación de dependencias desde requirements.txt

En esta captura se muestra el proceso de instalación de todas las dependencias necesarias para el proyecto mediante el comando `pip install -r requirements.txt`. La terminal comienza a descargar e instalar una serie de librerías relacionadas con modelos de lenguaje, ejecución en CPU y herramientas de machine learning.

Entre los paquetes que aparecen destacan componentes como `llama-cpp-python`, `transformers`, `torch`, `sentencepiece` o `bitsandbytes`, todos ellos esenciales para ejecutar y manipular modelos de IA. También se observan utilidades adicionales como `tqdm`, `einops` o `protobuf`, que complementan el entorno de trabajo.

```

((venv) aualaateca26@Mac-mini-de-aualaateca26 llama.cpp % pip install -r requirements.txt
Looking in indexes: https://pypi.org/simple, https://download.pytorch.org/whl/cpu, https://download.pytorch.org/whl/nightly, https://download.pytorch.org/whl/cpu, https://download.pytorch.org/whl/nightly, https://download.pytorch.org/whl/cpu, https://download.pytorch.org/whl/nightly
Ignoring torch: markers 'platform_machine == "s390x"' don't match your environment
Ignoring torch: markers 'platform_machine == "s390x"' don't match your environment
Collecting numpy~=1.26.4 (from -r /Users/aualaateca26/mi-chef-bot/llama.cpp/requirements/requirements-convert_legacy_llama.txt (line 1))
  Using cached numpy-1.26.4-cp311-cp311-macosx_11_0_arm64.whl.metadata (114 kB)
Requirement already satisfied: sentencepiece~=0.2.0 in /Users/aualaateca26/mi-chef-bot/venv/lib/python3.11/site-packages (from -r /Users/aualaateca26/mi-chef-bot/llama.cpp/requirements/requirements-convert_legacy_llama.txt (line 2)) (0.2.1)
Collecting transformers<5.0.0,>=4.57.1 (from -r /Users/aualaateca26/mi-chef-bot/llama.cpp/requirements/requirements-convert_legacy_llama.txt (line 4))
  Using cached transformers-4.57.6-py3-none-any.whl.metadata (43 kB)
Collecting gguf>=0.1.0 (from -r /Users/aualaateca26/mi-chef-bot/llama.cpp/requirements/requirements-convert_legacy_llama.txt (line 6))
  Using cached https://download.pytorch.org/whl/nightly/gguf-0.17.1-py3-none-any.whl.metadata (4.3 kB)
Collecting protobuf<5.0.0,>=4.21.0 (from -r /Users/aualaateca26/mi-chef-bot/llama.cpp/requirements/requirements-convert_legacy_llama.txt (line 7))
  Using cached protobuf-4.25.8-cp37-abi3-macosx_10_9_universal2.whl.metadata (541 bytes)
Collecting torch~=2.6.0 (from -r /Users/aualaateca26/mi-chef-bot/llama.cpp/requirements/requirements-convert_hf_to_gguf.txt (line 5))
  Using cached https://download.pytorch.org/whl/cpu/torch-2.6.0-cp311-none-macosx_11_0_arm64.whl.metadata (28 kB)
Collecting aiohttp~=3.9.3 (from -r /Users/aualaateca26/mi-chef-bot/llama.cpp/requirements/requirements-tool_bench.txt (line 1))
  Using cached https://download.pytorch.org/whl/nightly/aiohttp-3.9.5-cp311-cp311-macosx_11_0_arm64.whl (390 kB)

```

Paso 6: Descarga del modelo desde Hugging Face con Git LFS

En esta captura se muestra el proceso de preparación para descargar un modelo grande desde Hugging Face. Primero se inicializa Git LFS, una extensión necesaria para manejar archivos de gran tamaño, como los pesos de modelos de IA. Tras activarlo, se procede a clonar el repositorio del modelo Phi-3-mini-4k-instruct dentro de la carpeta de llama.cpp.

La terminal muestra cómo se descargan los archivos del repositorio: primero los metadatos y luego los ficheros pesados del modelo, que se transfieren mediante Git LFS. El proceso incluye la enumeración de objetos, la resolución de deltas y finalmente la descarga de varios gigabytes de datos.

```

((venv) aualaateca26@Mac-mini-de-aualaateca26 llama.cpp % git lfs install
Updated Git hooks.
Git LFS initialized.
((venv) aualaateca26@Mac-mini-de-aualaateca26 llama.cpp % git clone https://huggingface.co/microsoft/Phi-3-mini-4k-instruct
Clonando en 'Phi-3-mini-4k-instruct'...
remote: Enumerating objects: 111, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 111 (delta 1), reused 0 (delta 0), pack-reused 105 (from 1)
Recibiendo objetos: 100% (111/111), 1.03 MiB | 3.99 MiB/s, listo.
Resolviendo deltas: 100% (52/52), listo.
Filtrando contenido: 100% (3/3), 3.11 GiB | 24.01 MiB/s, listo.
((venv) aualaateca26@Mac-mini-de-aualaateca26 llama.cpp %

```


Paso 7: Descarga del modelo desde Hugging Face con Git LFS

En esta captura se muestra el proceso de preparación para descargar un modelo grande desde Hugging Face. Primero se inicializa Git LFS, una extensión necesaria para manejar archivos de gran tamaño, como los pesos de modelos de IA. Tras activarlo, se procede a clonar el repositorio del modelo Phi-3-mini-4k-instruct dentro de la carpeta de llama.cpp.

La terminal muestra cómo se descargan los archivos del repositorio: primero los metadatos y luego los ficheros pesados del modelo, que se transfieren mediante Git LFS. El proceso incluye la enumeración de objetos, la resolución de deltas y finalmente la descarga de varios gigabytes de datos.

```
[(venv) aulaateca26@Mac-mini-de-aulaateca26 llama.cpp % ls -d ../*/  
../llama.cpp/          ../Phi-3-mini-4k-instruct/    ../venv/  
[(venv) aulaateca26@Mac-mini-de-aulaateca26 llama.cpp % python convert_hf_to_gguf.py ../Phi-3-mini-4k-i  
nstruct --outfile ../chef-bot-v1.gguf --outtype q8_0  
INFO:hf-to-gguf:Loading model: Phi-3-mini-4k-instruct  
INFO:hf-to-gguf:Model architecture: Phi3ForCausalLM  
INFO:hf-to-gguf:gguf: loading model weight map from 'model.safetensors.index.json'  
INFO:hf-to-gguf:gguf: indexing model part 'model-00001-of-00002.safetensors'  
INFO:hf-to-gguf:gguf: indexing model part 'model-00002-of-00002.safetensors'  
INFO:gguf:gguf_writer:gguf: This GGUF file is for Little Endian only  
INFO:hf-to-gguf:Exporting model...  
INFO:hf-to-gguf:token_embd.weight,      torch.bfloat16 --> Q8_0, shape = {3072, 32064}  
INFO:hf-to-gguf:blk.0.attn_norm.weight,  torch.bfloat16 --> F32, shape = {3072}  
INFO:hf-to-gguf:blk.0.ffn_down.weight,   torch.bfloat16 --> Q8_0, shape = {8192, 3072}  
INFO:hf-to-gguf:blk.0.ffn_up.weight,     torch.bfloat16 --> Q8_0, shape = {3072, 16384}  
INFO:hf-to-gguf:blk.0.ffn_norm.weight,   torch.bfloat16 --> F32, shape = {3072}  
INFO:hf-to-gguf:blk.0.attn_output.weight, torch.bfloat16 --> Q8_0, shape = {3072, 3072}  
INFO:hf-to-gguf:blk.0.attn_qkv.weight,   torch.bfloat16 --> Q8_0, shape = {3072, 9216}  
INFO:hf-to-gguf:blk.1.attn_norm.weight,  torch.bfloat16 --> F32, shape = {3072}  
INFO:hf-to-gguf:blk.1.ffn_down.weight,   torch.bfloat16 --> Q8_0, shape = {8192, 3072}  
INFO:hf-to-gguf:blk.1.ffn_up.weight,     torch.bfloat16 --> Q8_0, shape = {3072, 16384}  
INFO:hf-to-gguf:blk.1.ffn_norm.weight,   torch.bfloat16 --> F32, shape = {3072}  
INFO:hf-to-gguf:blk.1.attn_output.weight, torch.bfloat16 --> Q8_0, shape = {3072, 3072}  
INFO:hf-to-gguf:blk.1.attn_qkv.weight,   torch.bfloat16 --> Q8_0, shape = {3072, 9216}  
INFO:hf-to-gguf:blk.10.attn_norm.weight, torch.bfloat16 --> F32, shape = {3072}
```

Paso 8: Configuración del modelo en la interfaz de ejecución

En esta captura se muestra la pantalla de configuración del modelo Chef Bot v1 dentro de la aplicación donde se va a ejecutar. Aquí se ajustan los parámetros que determinan cómo se cargará y funcionará el modelo una vez entrenado.

La interfaz muestra información sobre el consumo estimado de memoria, indicando que el modelo utiliza toda la memoria GPU disponible. También permite definir un identificador opcional para usarlo en llamadas a la API. Entre los ajustes visibles destacan el tamaño del contexto (4096 tokens), el nivel de offload a GPU, el número de hilos de CPU y el tamaño del batch para la evaluación.

Además, se incluyen opciones avanzadas como mantener el modelo en memoria, usar mmap, activar Flash Attention o ajustar parámetros experimentales de cuantización de caché. Esta pantalla sirve para afinar el rendimiento del modelo según las capacidades del equipo y las necesidades del proyecto antes de cargarlo definitivamente.

<

Chef Bot v1

Estimated Memory Usage

Beta

GPU 5.72 GB

Total 5.72 GB

API Identifier

Optionally provide an identifier for this model. This will be used in API requests. Leave blank to use the default identifier.

practica

Auto Unload If Idle (TTL)

☐

If set, the model will be automatically unloaded after being idle for the specified amount of time.

Context Length

?

4096

Model supports up to 4096 tokens

GPU Offload

?

32 / 32

CPU Thread Pool Size

?

7

Evaluation Batch Size

?

512

RoPE Frequency Base

?

☐

Auto

RoPE Frequency Scale

?

☐

Auto

Offload KV Cache to GPU Memory

?

☒

Keep Model in Memory

?

☒

Try mmap()

?

☒

Seed

?

☐

Random Seed

Flash Attention

?

☒

K Cache Quantization Type

?

☐

Experimental

V Cache Quantization Type

?

☐

Experimental

☐ Remember settings for chef-bot-v1

☒ Show advanced settings

Cancel

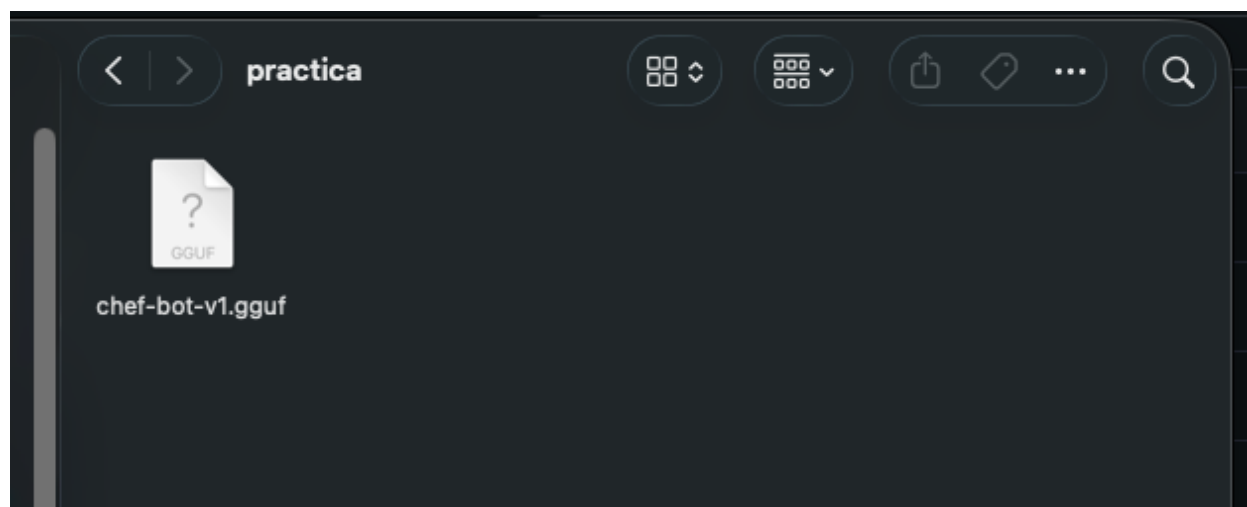
Load Model

⌘ ↵

Paso 9: Archivo final del modelo convertido a GGUF

En esta captura se muestra la carpeta donde se ha guardado el modelo ya convertido al formato GGUF, el estándar utilizado por herramientas como llama.cpp para ejecutar modelos de lenguaje de forma eficiente en CPU. Dentro de la carpeta aparece un único archivo: chef-bot-v1.gguf, que representa la versión final del modelo entrenado y preparada para ser cargada en la aplicación.

Este archivo es el resultado de todo el proceso previo: entrenamiento, ajuste fino y conversión. Es el modelo ya empaquetado y optimizado, listo para ser utilizado por el sistema que ejecutará el asistente culinario. La captura confirma que la conversión se ha completado correctamente y que el modelo está disponible en su ubicación final.



Paso 10: Prueba del modelo funcionando en la interfaz de chat

En esta captura se muestra el resultado final del proyecto: el modelo ya cargado y respondiendo dentro de la interfaz de chat. El usuario escribe una consulta culinaria sencilla —qué puede cocinar con garbanzos, espinacas y curry— y el asistente Chef Bot genera una receta completa basada en esos ingredientes.

La respuesta incluye un título, una lista de ingredientes y una breve explicación del plato, demostrando que el modelo ha aprendido a interpretar instrucciones y producir recetas coherentes en español. Esta captura confirma que el modelo entrenado, convertido y configurado previamente está funcionando correctamente y es capaz de interactuar de forma natural con el usuario.

